

FinSight-RAG

A Source-Grounded Question-Answering Assistant for Corporate Financial Filings

PHASE 3 — TECHNICAL REPORT

Lab 9 PSIS · Activity 1: RAG-Based Domain Assistant (Financial Report Analyzer) · SVKM's NMIMS MPSTME, B.Tech AI

Team: Akash Goyal (I025) — Data & Retrieval | Karan Lokhande (I039) — Application & Chain | Meer Magia (I040) — Evaluation, Report & Demo

GitHub: github.com/akashgoyalll/fin-sight-rag-1025-39-40 **Colab:** [FinSight_RAG_Phase3.ipynb](#) **Model:** gemini-3.5-flash

1. Problem and Use Case

Annual reports and SEC Form 10-K filings run to more than 100 pages and mix narrative sections (risk factors, management's discussion and analysis) with financial tables. Finding one figure means reading or searching the whole document. A general-purpose LLM asked about a specific filing answers from its training data and can state figures the filing does not contain. **FinSight-RAG** answers a question using only passages retrieved from the filing and returns the answer with the **document name and page number** of each supporting passage. If the retrieved passages do not contain the answer, it says so (`answerable = false`) instead of guessing. Users are students, analysts or investors who need a figure or statement they can verify in the source.

2. Document / Data Source

- **Filing:** NIKE, Inc. Form 10-K for the fiscal year ended May 31, 2023 (`nke-10k-2023.pdf`, 107 pages, 2.4 MB). It is public and is the example filing in LangChain's official RAG tutorial, which makes our results easy to reproduce. `scripts/get_data.sh` downloads it.
- **Content:** business overview, Item 1A risk factors, Item 7 MD&A (revenue, gross margin, segment results), financial statements and notes. All 107 pages have extractable text, so no OCR is needed.
- **Loading:** a small `pypdf` loader creates one LangChain Document per page with metadata source (file name) and page (1-based, as in a PDF viewer). It replaces `PyPDFLoader` from `langchain-community`, which was sunset in 2026.
- **Data-quality issue:** `pypdf` flattens tables into text. Column headers run together (e.g. `% CHANGE EXCLUDING CURRENCY CHANGES` on p. 42) and row/column alignment is lost, so questions that need numbers from a table (Q6) are harder than questions answered by a sentence.

3. Architecture and LangChain Components

FinSight-RAG — architecture (Phase 3 final system)

Every answer is returned with the document name and page of the supporting passage. Blue = LangChain component, orange = LLM, green = output, red tag = added in Phase 3.

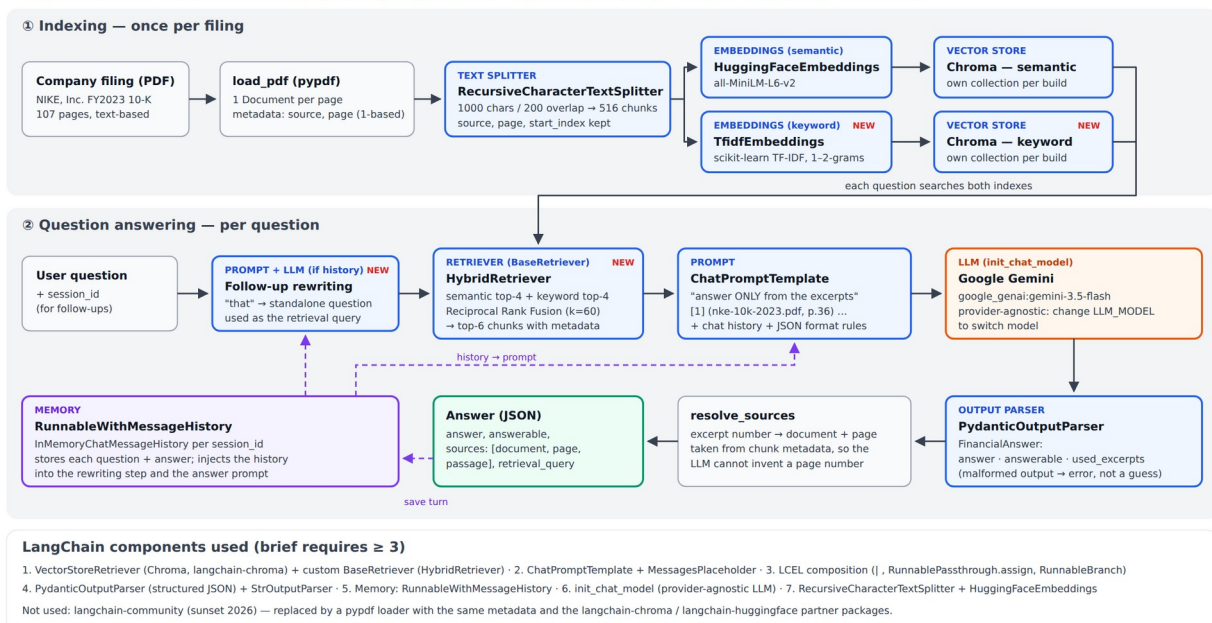


Figure 1. FinSight-RAG architecture. Indexing runs once per filing; each question runs one LCEL chain. Blue boxes are LangChain components.

The brief requires at least three LangChain components. We use: (1) **VectorStoreRetriever** over Chroma plus a custom **BaseRetriever** (**HybridRetriever**); (2) **ChatPromptTemplate** with **MessagesPlaceholder**; (3) **LCEL composition** (`|`, `RunnablePassthrough.assign`, `RunnableBranch`); (4) **PydanticOutputParser**; (5) **memory** via

RunnableWithMessageHistory; plus RecursiveCharacterTextSplitter, HuggingFaceEmbeddings and init_chat_model, which lets us switch LLM provider by changing one environment variable.

4. Chunking, Embedding and Retrieval Approach

- **Chunking:** RecursiveCharacterTextSplitter, 1,000 characters with 200 overlap, add_start_index=True, applied per page. This gives 516 chunks. Splitting per page means every chunk belongs to exactly one page, so each citation is a single page. The trade-off is that a table crossing a page break is split in two. The overlap keeps a sentence and the figure it describes together.
- **Embeddings and stores:** semantic embeddings from all-MiniLM-L6-v2 (384 dimensions, free, runs on a Colab CPU) and keyword vectors from TF-IDF (1-2-grams, 4,096 features, English stop words). Each set of vectors is stored in its own Chroma collection built from the same chunks.
- **Retrieval (Phase 3 change):** in Phase 2 the semantic retriever (top-4) and the keyword retriever failed on *different* questions, and together they found the evidence for all six. HybridRetriever takes the top-4 from each and merges them with **Reciprocal Rank Fusion**, $\text{score}(d) = \sum 1 / (60 + \text{rank of } d \text{ in each list})$ (Cormack et al., 2009). It then passes the best 6 chunks to the prompt. RRF uses only ranks, because cosine similarities from the two vector spaces are not on the same scale. A chunk found by both retrievers is counted once and ranked higher.
- **Generation and citations:** the prompt lists the chunks as numbered excerpts labelled [n] (document, p. N) and tells the model to answer only from them. Gemini (gemini-3.5-flash via init_chat_model; no temperature is set, following Google's Gemini 3 guidance) returns JSON {answer, answerable, used_excerpts}, which PydanticOutputParser validates. Citations are built from the metadata of the excerpts the model names, so the model cannot invent a page number. Excerpt numbers that do not exist are dropped.
- **Conversation:** RunnableWithMessageHistory keeps the history for each session. When there is history, the LLM first rewrites the follow-up into a standalone question, and retrieval uses that rewritten question (tested in Section 5.4).

5. Testing and Evaluation

Method. Seven questions (tests/questions.json), each with a ground truth we checked by hand against the PDF text. They cover a numeric lookup, a fact, a number with an explanation, a narrative risk question, a multi-hop calculation designed to be hard (Q6: segment EBIT margins are not stated in the filing and must be computed from two columns of the p. 39 table) and one question the filing cannot answer (Q7). We evaluated at three levels. (a) **Retrieval:** is the evidence string, and is a ground-truth page, among the retrieved chunks? (b) **Correctness:** does the answer match the ground truth? (c) **Grounding (manual review):** is every figure and claim in the answer present in, or directly computed from, the passages it cites? We rate grounding as Yes / Partial / No. Automatic string checks are recorded by scripts/run_phase3.py; the correctness and grounding columns below come from our reading of each answer against its cited passages.

5.1 Retrieval: Phase 2 baseline vs hybrid (Q1-Q6)

Retriever	Q1	Q2	Q3	Q4	Q5	Q6	String hits	Page hits
Semantic top-4 (Phase 2)	HIT	miss (pX)	HIT	HIT	miss	HIT	4/6	5/6
Keyword top-4	miss (pX)	HIT	HIT	HIT	HIT	miss	4/6	5/6
Semantic top-6	HIT	miss	HIT	HIT	miss	HIT	4/6	6/6
Keyword top-6	miss (pX)	HIT	HIT	HIT	HIT	miss	4/6	5/6
Hybrid RRF top-4	HIT	HIT	HIT	HIT	HIT	HIT	6/6	6/6
Hybrid RRF top-6 (final)	HIT	HIT	HIT	HIT	HIT	HIT	6/6	6/6

Table 1. Retrieval for Q1-Q6 in Colab (all-MiniLM-L6-v2 + TF-IDF). HIT = the evidence string is in the retrieved chunks; (pX) = no ground-truth page retrieved. Page hits count a ground-truth page anywhere in the results.

Hybrid retrieval found the evidence for all six answerable questions at both k = 4 and k = 6 (6/6 by evidence string and by page). Either retriever alone found 4/6. Giving a single retriever six chunks instead of four did not fix its misses. Semantic top-6 reaches a page-9 chunk, but not the one with the employee count. Keyword top-6 still misses Q1 and the revenue table needed for Q6. So the gain comes from combining the two rankings, not from a larger context. The semantic Q5 miss is the string-check false negative we reported in Phase 2: the retrieved p. 46 does discuss NIKE's currency exposures.

5.2 Generated answers of the final system and grounding review

Q	Question (short)	Answer (abridged)	Cited pp.	Correct?	Grounded?
Q1	Total revenues FY23	\$51.2 billion (\$51,217 million).	36, 89, 31	Yes	Yes

Q	Question (short)	Answer (abridged)	Cited pp.	Correct?	Grounded?
Q2	Employees at May 31, 2023	Approximately 83,700 employees worldwide, including retail and part-time employees.	9	Yes	Yes
Q3	Gross-margin change	Decreased 250 bps to 43.5% from 46.0%; lists the four drivers (product costs, NIKE Direct promotions, FX incl. hedges, off-price margin).	37	Yes	Yes
Q4	NIKE Direct growth	+14% (+20% currency-neutral), from \$18.7 billion to \$21.3 billion.	36, 31	Yes	Yes
Q5	FX-rate risks	Four risks: lower revenue and margins; translation of foreign-currency sales; product/inventory cost exposure; disruption to independent manufacturers.	46, 14, 82	Yes (relevant)	Yes
Q6	Highest segment EBIT margin (weak case)	Computes all four margins: Greater China 31.50%, APLA 30.04%, EMEA 26.31%, North America 25.24% → Greater China.	39	Yes	Yes (computed from p. 39)
Q7	FY2025 revenue (not in filing)	The excerpts do not contain FY2025 revenue; data only goes up to FY2023 (answerable = false).	—	Yes (declined)	n/a

Table 2. Final system (hybrid top-6, gemini-3.5-flash), one run in Colab on 1 Oct 2026. Correct = matches the ground truth. Grounded = every figure or claim is found in, or computed from, the full text of the cited chunks (checked by hand).

All seven answers of the final system were correct, and every figure in them appears in a passage it cites. With the Phase 2 retriever (same prompt and model), six of seven were correct. Q2 failed because the employee count was not retrieved (Section 5.3). Both systems declined Q7 (answerable = false) instead of inventing a FY2025 figure. The model also added correct, cited context it was not asked for, e.g. the 20% currency-neutral growth in Q4 and the four margin drivers in Q3.

5.3 Where the system performs poorly

Retrieval failure: Q2 with the Phase 2 retriever. The semantic retriever returned pp. 4, 86, 6 and 36 for the employee question. None contains the figure, which is one sentence on p. 9 ("approximately 83,700 employees worldwide"). The model therefore said the excerpts do not contain the information (answerable = false). This is the safe way to fail, since no number was invented, but it is still a wrong answer to an answerable question. A sentence embedding represents a short exact fact weakly, while TF-IDF matches "employees" and "May 31, 2023" directly. Hybrid retrieval fixed it: p. 9 is retrieved and the answer is correct.

Keyword failures: Q1 and Q6. TF-IDF ranked the segment tables headed "TOTAL REVENUES" (pp. 40–43) above the sentence stating NIKE, Inc. revenues. For Q6 it found the EBIT table but not the revenue table. Some of these chunks still reach the final context (pp. 41–43 for Q1). The model ignored them here, but they are noise.

The designed weak case (Q6) did not fail. We expected the model to name North America (largest absolute EBIT) or to decline, because segment margins are not stated in the filing. Instead, Gemini 3.5 Flash read EBIT and revenue for all four segments from the flattened p. 39 tables and computed the margins correctly (31.50%, 30.04%, 26.31%, 25.24%) in both systems. We count this as a pass that depends on the model doing arithmetic in free text. Nothing verifies the division, and one run does not show it is always right (Section 7).

5.4 Multi-turn test (memory)

We asked Q1 and then, in the same session, "How much did that grow on a currency-neutral basis?". With rewriting, the retrieval query became "How much did NIKE, Inc.'s total revenues grow on a currency-neutral basis in fiscal 2023?" and the answer was 16%, which is correct and grounded. Without rewriting (Phase 2 behaviour), the raw follow-up was searched and the answer was also 16%, because "currency-neutral" is a distinctive term that appears next to the revenue figure. The test confirms that memory works (4 messages kept after two turns) and that rewriting produces a correct standalone query. It does not show a difference in the answer. A follow-up without a distinctive keyword (e.g. "And the year before?") would be a stronger test.

5.5 Is the output grounded in the source?

Yes, for this test set. In the final system every factual statement in Q1–Q6 is found in, or computed from, the cited chunks. Page numbers are correct by construction, because they come from chunk metadata. The model declined whenever its excerpts lacked the answer (Q7, and Q2 with the baseline retriever), so we saw no invented figures. There are three caveats. Grounding was judged by us, on seven questions, from one run. The automatic `cited_ok` check only confirms that a cited page is a ground-truth page, not that each sentence is supported. Q5 is the least constrained answer (a summary of four risks), but each of its four points matches p. 14 or p. 46.

6. Challenges Encountered and How We Solved Them

- **Semantic search missed exact facts.** In Phase 2 the semantic retriever never returned the p. 9 sentence with the employee count, so the system answered Q2 with "the excerpts do not contain this" even though the filing does. TF-IDF found it, but TF-IDF failed on Q1 and Q6. We combined both with Reciprocal Rank Fusion (Section 4). Evidence retrieval went from 4/6 to 6/6, and Q2 is now answered correctly from p. 9 (Section 5).
- **Free-tier API limits.** Our Gemini key allowed 5 requests per minute and 20 per day per model. The first evaluation run hit the per-minute limit: two baseline answers took 76 s and 140 s while the client retried. The live-demo cells then hit the daily limit. We changed `run_phase3.py` to space calls 13 s apart, wait for the retry delay that Google returns, save after every answer, and resume without re-asking questions already answered.
- **Duplicate chunks in the index.** Chroma's in-process clients share state. Building the index twice in one Python session (re-running a notebook cell, or building the semantic and keyword indexes) appended to the same collection: 516 chunks became 1,032 and results contained duplicates. Each index now gets its own collection name, and a regression test checks the count stays at 516.
- **Deprecated libraries.** `langchain-community` (PyPDFLoader, BM25Retriever) was sunset in 2026. We wrote a small `pypdf` loader with the same metadata, used the `langchain-chroma` and `langchain-huggingface` partner packages, and wrote the hybrid retriever as our own `BaseRetriever`.

7. Limitations and Proposed Improvements

- **Arithmetic over flattened tables.** Q6 passed, but only because the model divided numbers it read from `pypdf`'s flattened table text; nothing checks the calculation. *Improvement:* table-aware parsing (rows and columns) plus a calculator tool, so ratios are computed, not generated.
- **Small, single-run evaluation.** One filing, 7 questions and one run of the model, judged by the team. The results are indicative, not statistically meaningful. *Improvement:* several companies and years with `company/year` metadata filters, a larger question set (e.g. FinanceBench-style), repeated runs and an automatic faithfulness metric.
- **Noisy context.** Hybrid top-6 also brings in irrelevant chunks (e.g. the segment tables on pp. 41–43 for Q1). $k = 6$, the RRF constant and equal weights were fixed in advance, not tuned, and the keyword side is TF-IDF, not BM25. *Improvement:* a cross-encoder re-ranker, and tuning k on held-out questions.
- **Memory and API.** `RunnableWithMessageHistory` works but is deprecated in `langchain-core` 1.6, memory lives only in the Python process, and the free API tier limits throughput (20 requests per day per model). *Improvement:* move memory to a LangGraph checkpoint, and use a paid tier or local model for larger evaluations.

8. Team Contributions

Roles as set in Phase 2. **Akash Goyal (I025), Data & Retrieval:** data source, loading, chunking, embeddings, vector stores, hybrid retrieval, repository. **Karan Lokhande (I039), Application & Chain:** LCEL chain, prompt, output parser, memory and follow-up rewriting, LLM integration, notebook. **Meer Magia (I040), Evaluation, Report & Demo:** test questions and ground truth, grounding review, architecture diagram, report, video. Individual contribution statements are submitted separately.

References

- [1] NIKE, Inc. (2023). Form 10-K for the fiscal year ended May 31, 2023. U.S. Securities and Exchange Commission.
- [2] LangChain documentation: RAG tutorial, LCEL, retrievers, structured output, message history. docs.langchain.com.
- [3] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. SIGIR '09.
- [4] Reimers, N., & Gurevych, I. (2019). Sentence-BERT. EMNLP-IJCNLP (all-MiniLM-L6-v2 model family).
- [5] Google (2026). Gemini API models and Gemini 3 developer guide. ai.google.dev.